

06 AI 能在自己的世界模型里学习吗？

作者 NILUSHANAN KULASINGHAM

阅读约 13 分钟 · 可交互

如果练习发生在模型里面，一个月的机器人时间就变成一天。从 Dyna 起这就是卖点，而 Dreamer 让它真的成立。这笔汇率的代价是什么，以及为什么「智能体在利用自己的梦」的解法是故意把梦弄差。

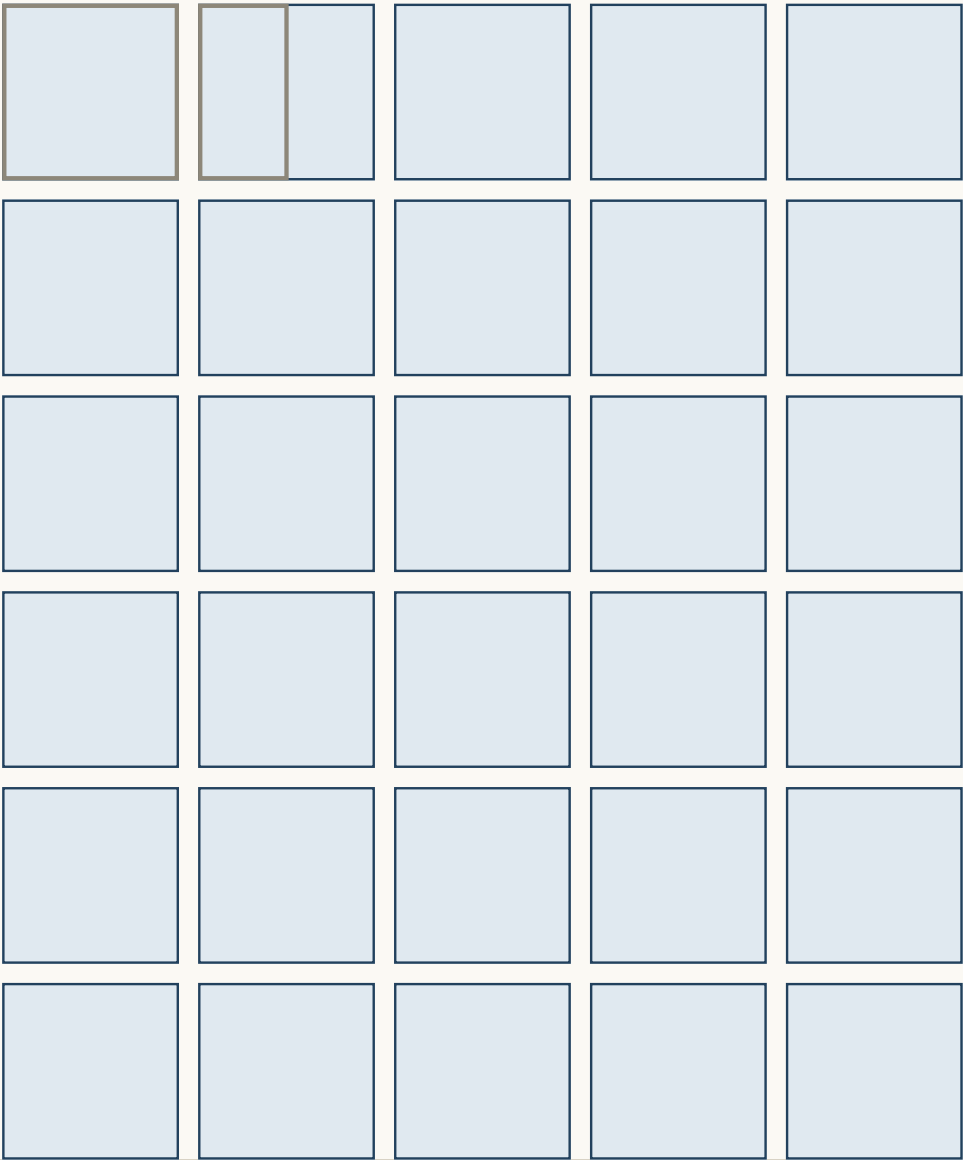
本章的图都是可交互的。请到 worldmodels101.com/zh/chapters/dreaming 在线按一按、拖一拖。

一条机械臂学会把东西拿起来之前，会先把它掉在地上几十万次。每一次掉落都要付出真实的代价：挂钟上的秒数，还有关节的磨损。它把箱子撞翻的时候，旁边还得有个人过来把场景摆回去。按一秒一次算，这就是一个月里什么别的都不干。

这个想法一句话就能说完。拿那个月里的一部分时间去收集经验，用这些经验拟合一个模型，然后让机械臂在模型里面练，因为在那里掉一次东西只花算力。Richard Sutton 在 1991 年把这个循环写了下来，并把那个智能体叫作 Dyna。Sutton 是强化学习的奠基人之一，强化学习讲的是智能体靠行动、靠拿到奖励来学习，那本标准教材也是他合写的。

Dyna 很小，只是一个跑在格子迷宫里的循环，而且早了大概三十年。Danijar Hafner 的 Dreamer 用深度网络跑同一个循环，2019 年才出现。标题里那个问题问的是这个循环行不行，简短的回答是行。长一点的答案讲的是它要付出什么。

模型从这里拟合



机械臂在跑的天数

30.0

去扶箱子的趟数

500

总挂钟时间

30.0 天

FIG. 6.1 开头说的那一个月，画成机械臂在世界上掉东西的三十天。把练习拖进模型里，看着这些天从月末往前一格一格空掉。有两样东西不会动：最前面那一天半，因为模型总得拟合在什么东西上；还有那些去扶箱子的趟数，每一趟都得有人走过去。这幅图完全没有给「一步想象值多少」定价，那是下一个问题。这里的代价数字仅作参考。

把大部分练习挪进去，你应该会看到这一个月降到大约一天半。从 Dyna 到 Dreamer，卖的一直是这个，而且这笔节省是真的。它只在「想象出来的一步仍然有用到值得计数」的时候才成立。

简短的回答是行，而 Dreamer 就是证据

在模型里学习，是「造一个模型」这件事最大的实际理由，而 Dreamer 就是证据。Danijar Hafner 在多伦多读博期间、在 Google Brain 里面把它做了出来，那是他在的 PlaNet 用搜索在一个学出来的模型里做规划之后一年。Dreamer 丢掉了搜索，改成训练一个策略 (挑选动作的那一部分。这里被训练的是它，不是模型)。

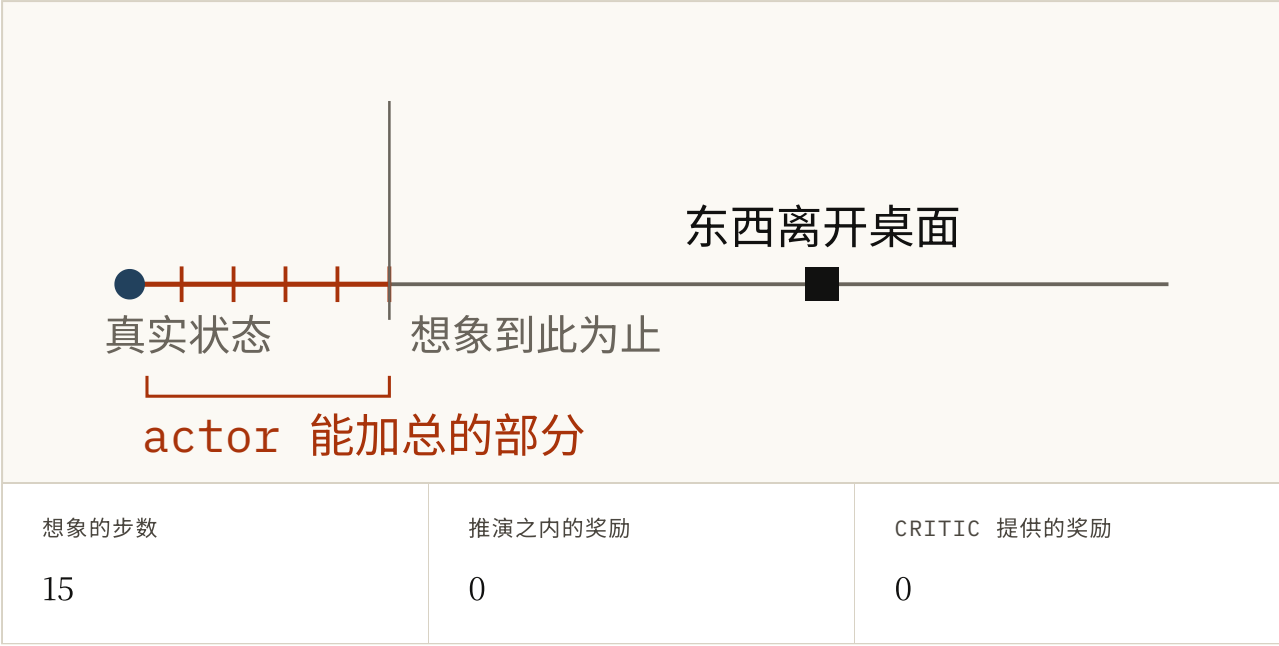


FIG. 6.2 actor 是从一段想象推演里学的，而这段推演早在机械臂把东西拿离桌面之前就停了。把奖励往外拖，看着它离开想象的那一段，于是 actor 就没有什么可以加总的了。然后把 critic 打开：它给 actor 递过去一个数，代表最后一步想象之后的全部。那个数是猜的，其中没有任何一点来自世界。这里的长度仅作示意。

2019 年那篇 *Dream to Control*，是从很长的想象推演里学出行为的。学习期间，actor 和 critic 谁都没有碰过环境。actor 就是策略换了个名字。critic 是第二个网络，它猜的是前面还有多少奖励。

DreamerV2 在 2020 年跟上，论文是 *Mastering Atari with Discrete World Models*。所谓离散，是指状态由一组多选题的答案构成，而不是一组旋钮。在 Atari 游戏上，它开始赢过那些直接从环境里学的智能体。

DreamerV3 在 2025 年发表于 *Nature*，标题是 *Mastering diverse control tasks through world models*。它用同一套设置跑了 150 多个任务。它是第一个不靠人类数据就在 Minecraft 里挖到钻石的智能体，而这需要一长串按顺序完成的步骤。这是我知道的、对「在想象中学习能不能泛化」这个问题最有力的回答。

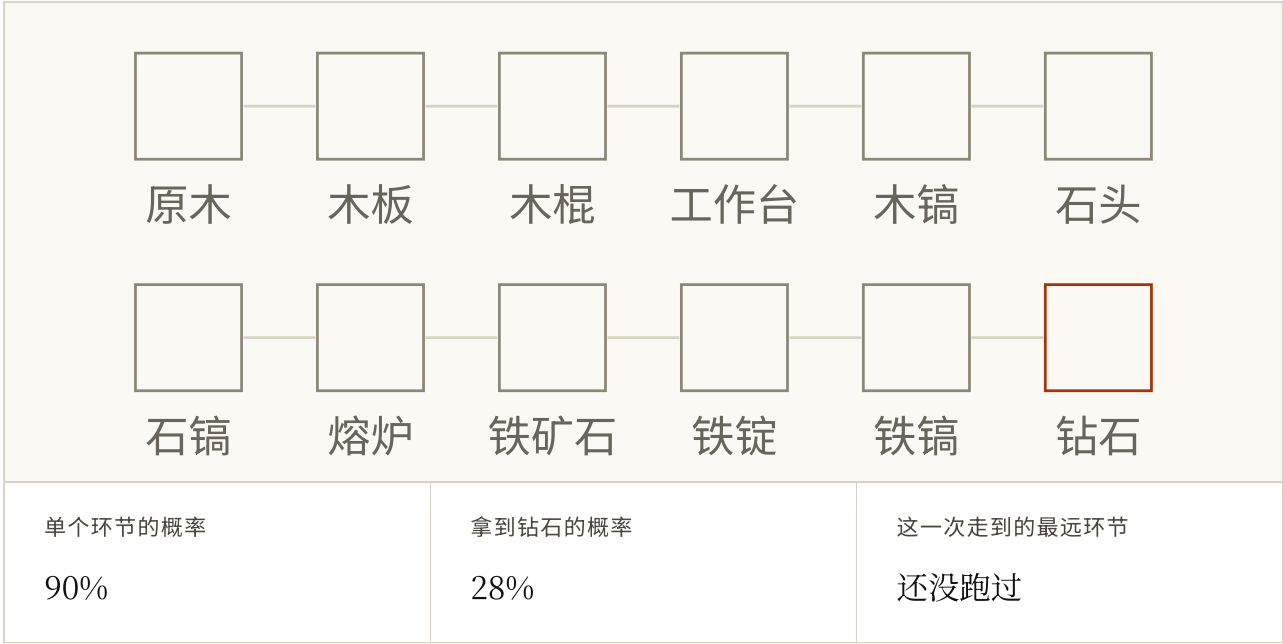


FIG. 6.3 钻石是按顺序走完的十二步，而 DreamerV3 是在没人教它怎么走的情况下走到的。拖动单个环节的成功概率，再按「跑一次」，看看这条链在哪儿断掉。每个环节十次里成九次，整条链还是十次里要失败大约七次，所以走完这条链作为证据，比任何单个任务都值钱。单环节的概率仅作示意；那十二个环节是游戏本身的。

所以这类系统只用「直接学习」所需的一小部分世界接触，就能达到胜任水平。而在一台真实机器人上，这一小部分决定的是学得成还是学不成。那一个月掉在地上的东西，正是这条家族线要缩短的。现在我们来看看这个循环到底在做什么，因为代价就藏在那儿。

这个循环必须转两圈

Dyna 的循环有四步，而且会重复。在世界上行动一阵子，把发生的事留下来。用它拟合一个模型。在那个模型里训练策略，这一步花的是算力，不是机器人接触，然后把行为带回世界。

带回来的这个行为，收集到的经验比上一圈的好。这让下一个模型更好。下面这幅图里，第二圈才是要盯着看的部分。

循环再转一圈这件事，是摘要里总会被丢掉的。一个拟合在「未经训练的智能体乱扑腾」之上的模型，只在未经训练的智能体会去的地方管用。它要等到一个更好的策略走到新地方去，才会变好。

1 在世界中行动 2 拟合模型 3 在模型中训练策略 4 把行为带回世界

目标



9

起点

0

96 格中的 0 格

否

FIG. 6.4 一个小小的格子世界，目标在远处的角上。按「跑一圈」。第一个策略是随机乱走，所以模型只学到了起点附近的那些格子，也就是阴影的那些，在它里面训练出来的策略根本不知道目标在哪儿。再按一次。更好的策略走得更远，阴影区域变大，一两圈之后模型终于覆盖到了目标。然后按「重置」，试着把「改进策略」关掉：阴影区域就不再长了，一圈又一圈。全都仅作参考。

第一个模型是拟合在「一个没用的策略碰巧撞上的东西」上的。它知道地板，对目标一无所知。第二圈把这个补上了，因为更好的策略会去新的地方，把第一个模型缺的那些数据带回来。

这也是为什么图 6.1 里那一天半是锁住的。想象出来的步数只有在模型能准确供得上时才值钱，而模型只在它去过的地方才做得到这一点。给机器人世界准备的网络，对其余的地方只能靠猜。

想象经验是另一种货币

训练日志会把一次真实转移和一次模型生成的转移记成两行。转移就是一步经验：一个动作，以及它导致了什么。学习者不能这样算。

想象出来的经验带着模型的盲点。推演，也就是一串想象出来的转移，每从真实状态往外走一步，这份折扣就再累积一层。我把这个叫作汇率。

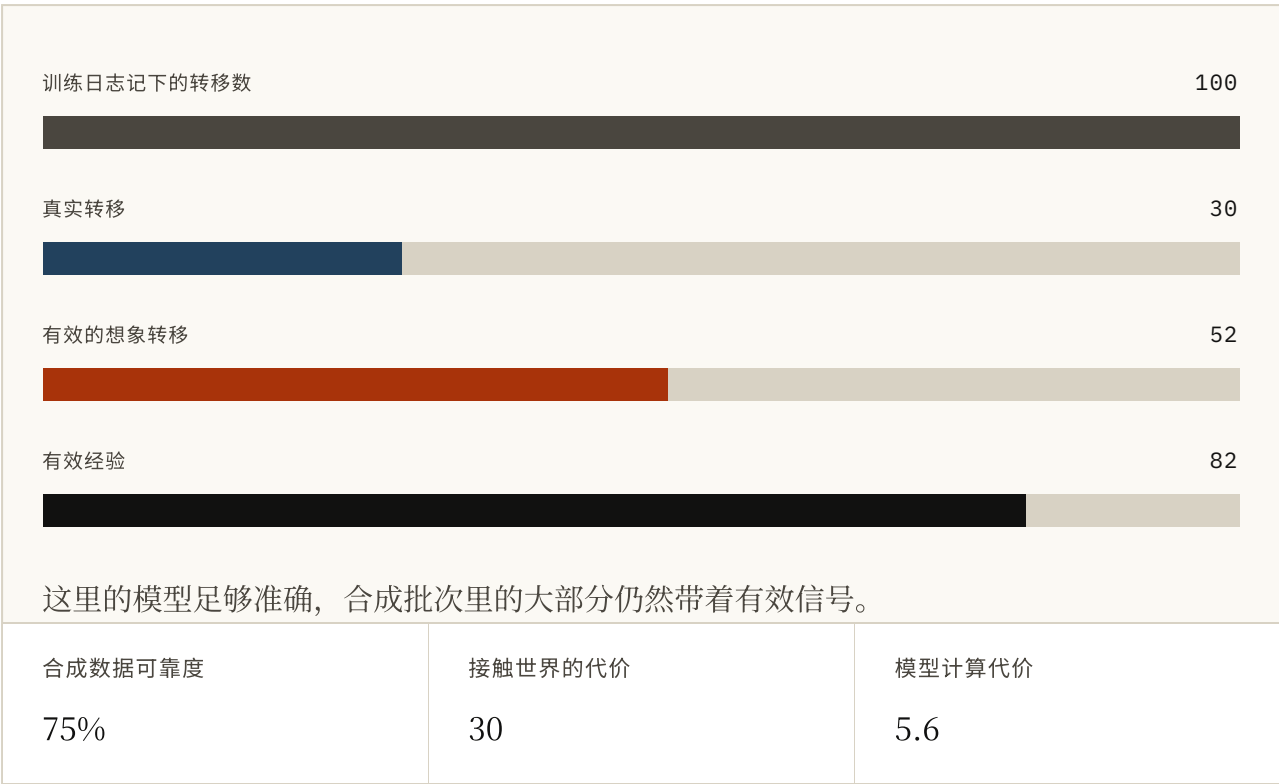


FIG. 6.5 把想象数据占比往上推，再把每步模型误差往上推。名义上这一批仍然是 100 次转移。看着「有效经验」那根条往下掉，那才是学习者真正拿到的东西。这个折扣是示意，不是估计。它暴露出来的，是「生成的行和观测的行可以按面值互换」这个假设。

Michael Janner 等人在 2019 年给它定了价。他们那篇 *When to Trust Your Model* 提出了 MBPO，它只在很短的想象推演上训练策略。每一段都从回放缓冲区里抽出的一个真实状态出发，回放缓冲区就是行动时攒下来的那批真实经验。

这是拿数量去换可信度。只有当每一步都按它离证据多远来定价时，多想象才有用。

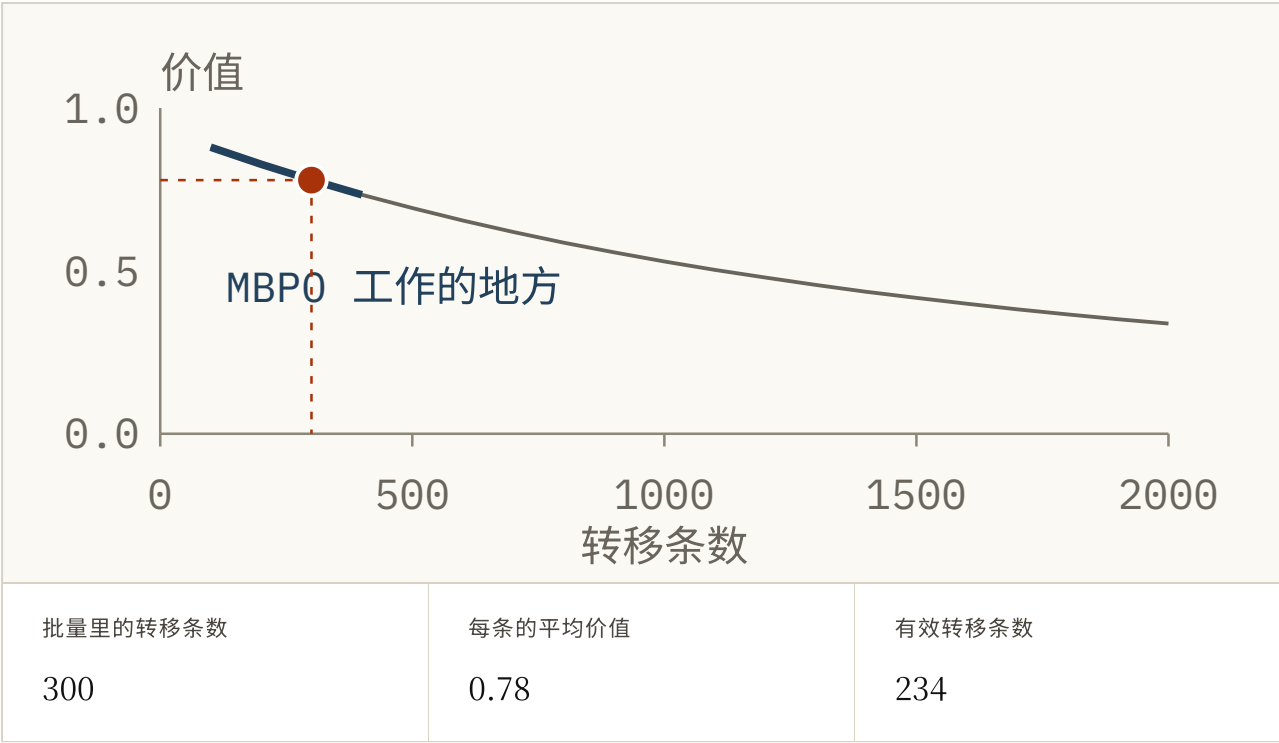


FIG. 6.6 回放缓冲区里的一百个真实状态，以及「从每一个往外想多远」这一个选择。拖动推演长度，看着那个点沿曲线往下走：批量变大了，而其中仍然贴着证据的那一份变小了。MBPO 工作在左上角那个角落里，从每一个真实状态往外只走一两步。一边拖一边看第三格，因为只有那一格算数。可信度数字仅作示意；这笔交换是论文的。

火球策略

一个在模型里训练出来的策略，是被那个模型打分的。模型就是它能看到的全部，所以它没办法把一个好看动作，和一个只在批改它的模型眼里好看的动作分开。它会去找后面那一种，因为那些是眼前最便宜的分数。

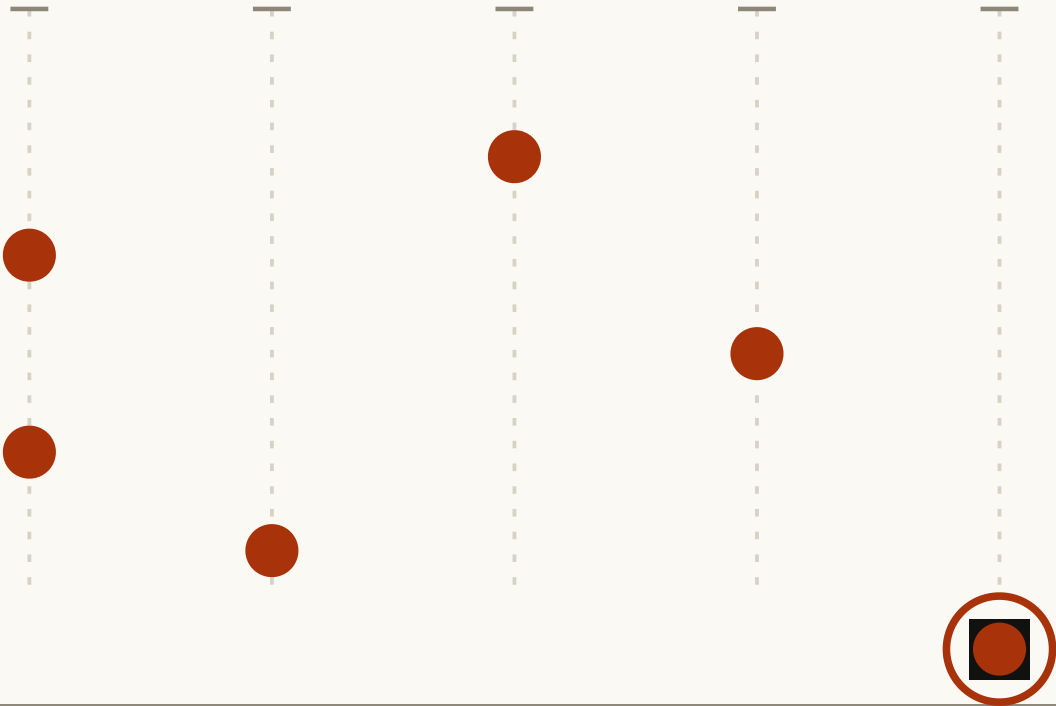
David Ha 与 Jürgen Schmidhuber 在 2018 年的 *World Models* 里正面撞上了这件事。Schmidhuber 从 1990 年起就在主张要学出世界模型，他还是 LSTM 的共同发明人，那是撑起十年语音识别的记忆网络。Ha 当时在 Google Brain，把这项工作做成一个可交互的网页发了出来。

他们的智能体是一个很小的控制器，完全在一场 Doom 关卡的梦里训练出来，玩家在那一关里要躲火球。这个控制器找到了一种移动方式，能让梦干脆不再产生火球。我们来看看那是什么样子。

梦里

策略：躲避

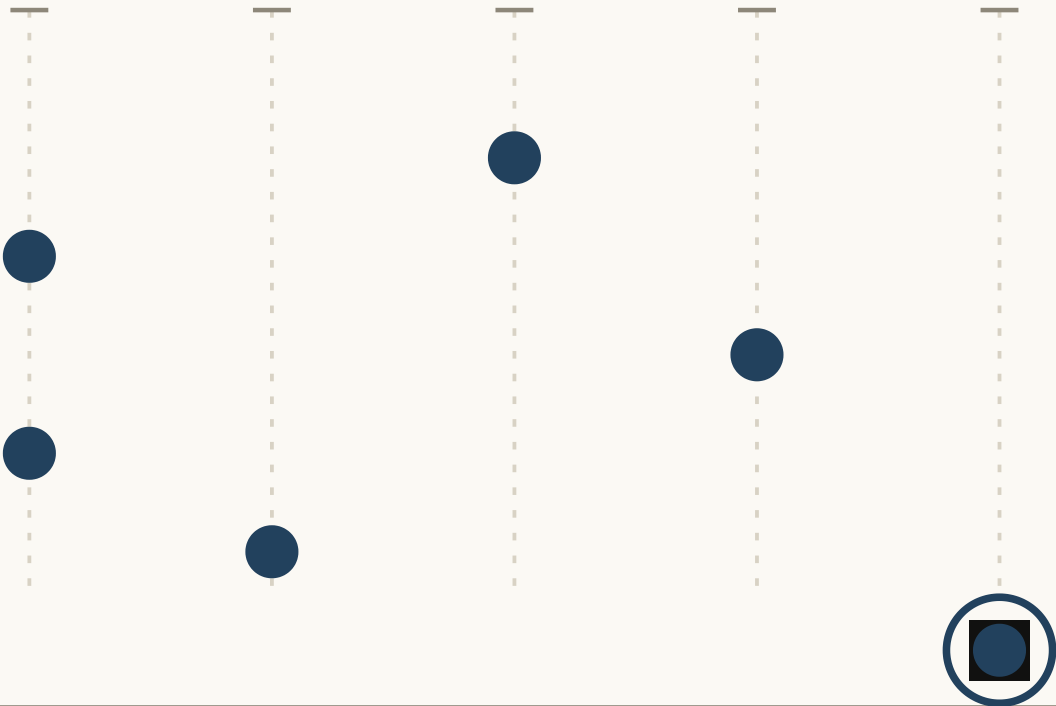
第 15 步，共 20 步 · 被击中



真实游戏

策略：躲避

第 15 步，共 20 步 · 被击中



梦里的得分

14 / 20

真实游戏的得分

14 / 20

生成的火球：梦 / 真实

14 / 14

FIG. 6.7 按「在梦里训练」，看着分数往上爬。然后看火球：智能体找到了一种走法，让梦不再生成火球，所以它在里面的分数近乎满分。现在按「在真实游戏里运行」。同样的动作，火球照样来，因为游戏从没答应过停火。这里的一切都是玩具，但这个发现是 Ha 与 Schmidhuber 的。

在模型里面，这是个出色的策略。在真正的游戏里，它一文不值，因为游戏并没有同意停火。火球策略就是那个找到了批改人盲点的学生。

规划器那个版本更容易抓住，而我把这一点算在规划这边。一个在决策时做搜索的规划器可以被否决，因为方案就摆在那儿可以查。而一个在梦里训练出来的策略，走出来的时候已经把那个漏洞带在自己的权重里了。

把梦弄得比世界还难

Ha 与 Schmidhuber 的解法是一个故意更差的模型。如果策略在利用某个怪癖，那就让这个怪癖变得不可靠。

在训练时把模型预测里的不确定性调高 (这个旋钮通常叫温度。低表示模型只把它最可能的那一个猜测交给你。高表示它采样得更散)。于是同一个动作不再总是产生同一个方便的结果，而一个需要梦往某个方向掷才管用的把戏，也就不值得再往上搭房子了。

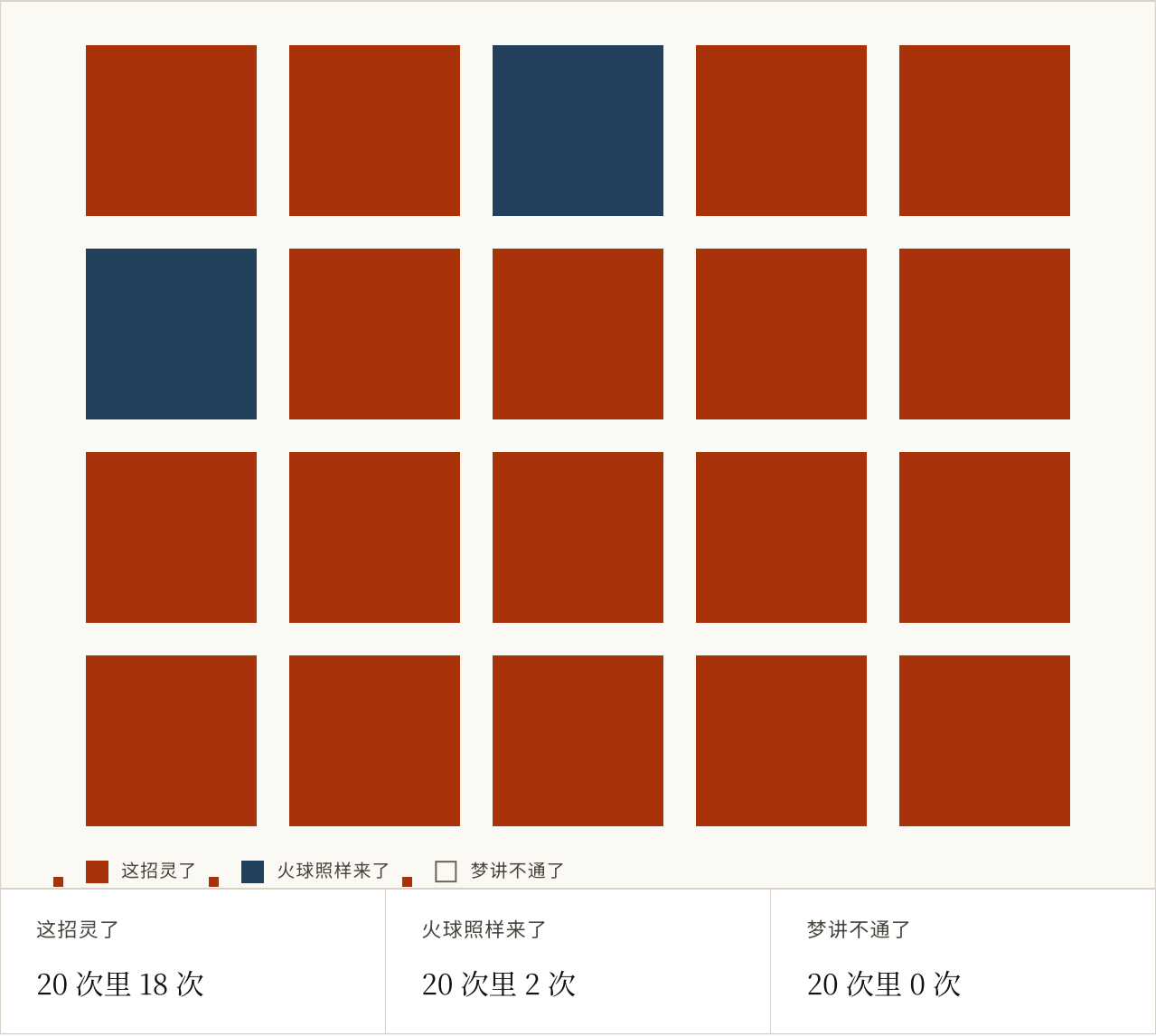


FIG. 6.8 同一个把戏跑二十次，就是「待在角落里就不再有火球」那一招。拖动温度，再按「再掷一次」。在最左边，这招每次都灵，所以策略可以整个搭在它上面。在中间，它偶尔才灵，这就不够了。在最右边，什么都不可靠了，游戏本身也一样，于是里面已经没有任务可学。全都仅作参考，不过这个发现是 Ha 与 Schmidhuber 的。

如你所见，这个旋钮的两端都会失败。一个笃定的梦会被利用，一个噪声太大的梦里已经没有任务可学。管用的是中间那条窄带，在那里两种失败都还没占上风。

机器人领域早一年就做过同样的动作。模拟器是一个手工搭出来的模型，也有自己的怪癖。OpenAI 的 Josh Tobin 等人把这个解法叫作域随机化（domain randomisation），写在 2017 年一篇同名论文里。颜色、光照和相机角度都随机变化，于是模拟器的变化比现实还大。

在那里学到的任何东西，都不可能依赖现实的某一个版本。OpenAI 2019 年的论文 *Solving Rubik's Cube with a Robot Hand* 把这个想法推到了真实硬件上。它的随机化范围会随着策略变好而自己放宽。

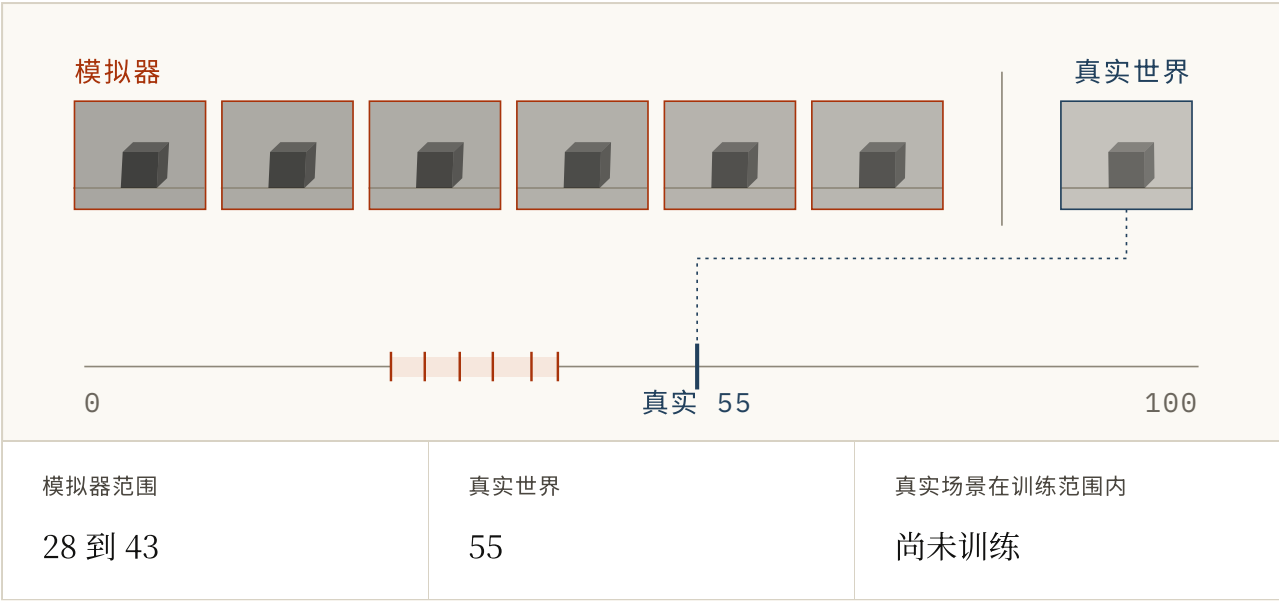


FIG. 6.9 模拟器把同一个场景画了一遍又一遍，每次改变颜色和相机，而真实世界是右边那个固定的样本。把变化留在低位再按「训练」：策略会靠着它见过的那个确切颜色和那个确切相机，而真实场景落在外面。把变化调到超过真实场景再训练一次，现在它就落在那条带子里面了。打开「边学边放宽」，就能看到魔方那个版本。全都仅作示意。

这是件挺奇怪的事。你把整份预算都花在把模型做准上，然后又把它弄糊，好让任何东西都不能靠在这份准确上。两个领域前后不到一年，都走到了这一步。

这条家族线没买到什么

所以这个循环是管用的，我们也看到了它的代价。最后我想说三件 Dreamer 这条线没有买到的事，因为每一件都回到那个汇率上。

它没有消除对真实经验的需要。它改变的是汇率。总得有东西出去弄清楚实际会发生什么，而模型也只在那件事被做过的地方才值得信任。

它没有修好打分这件事。策略自始至终都由模型批改。短推演、温度旋钮和随机化的模拟器，都只是让这份批改更难被钻空子，没有一个能让它变正确。

还没有分数		
模型打的分数	世界打的分数	检验花掉的真实步数
0	0	0

FIG. 6.10 给策略的一张成绩单。按「再训练一轮」，就多出一个分数，是模型打的，不花钱。按「拿到世界里检验」，多出来的是一个石板蓝的分数，要花两千步真实经验，而且分数低得多。看看那一列朱红色能排多长，才终于有东西来检验它。这些分数仅作参考。

它没有让迁移变免费。一个在梦里管用的策略，在有人把它拿到外面跑之前，只是一个关于世界的、尚未检验的主张。Ha 与 Schmidhuber 跑了，发现火球照样来。

希望这一章帮上了忙。下面有一个简短的测验，你可以用它检查这些想法有没有留下来。如果我哪里写错了，或者你知道比我用的更好的来源，「关于」页面有一节讲勘误。我很乐意收到你的来信。

试一试

1 在模型里面训练，改变的是哪种资源的汇率？

- a. 所需的参数量
- b. 内存，它变便宜了
- c. 和世界的接触；其中一部分改由模型计算来付
- d. 模型精度，它免费变好了

答案 c. 和世界的接触；其中一部分改由模型计算来付. 学习者仍然需要它那份经验。变的是这些步从哪儿来，以及由哪份预算付账。

2 一个仪表盘把 90 次想象转移和 10 次真实转移记成 100 个训练样本。这本账漏了什么？

- a. 要按模型误差以及离真实状态多远，给可靠性打折
- b. 策略的参数量
- c. 两种来源必须使用同样的批量大小
- d. 想象转移没法保存

答案 a. 要按模型误差以及离真实状态多远，给可靠性打折. 对日志来说每一行都一样，对学习者不是。合成经验继承模型的盲点，而且这笔债会沿推演累积。

3 循环的第二圈修好了第一圈修不了的什么？

- a. 它去掉了解码器的需要
- b. 它缩短了推演
- c. 它把模型变小了
- d. 更好的策略会去新的地方，产生第一个模型缺的那份数据

答案 d. 更好的策略会去新的地方，产生第一个模型缺的那份数据. 拟合在「未训练的智能体乱扑腾」之上的模型，只在那种智能体会去的地方管用。模型变好是因为策略变好，反过来也一样。

4 一个在模型里面训练的策略，是被什么打分的？

- a. 世界
- b. 模型，而且整个训练过程中只有模型
- c. 来自真实环境的一份留出测试集
- d. 一位人类评估者

答案 b. 模型，而且整个训练过程中只有模型. 它在训练期间接触不到世界，所以没法把「真正好的动作」和「只是在批改者眼里好看的动作」分开。

5 David Ha 与 Jürgen Schmidhuber 的智能体找到了一种移动方式，让它的梦不再产生火球。这是哪一类失败？

- a. 探索不足
- b. 训练代码里的 bug
- c. 策略在最大化模型交给它的分数，而这正是它被要求做的事
- d. 模型太小了

答案 c. 策略在最大化模型交给它的分数，而这正是它被要求做的事。什么岔子都没出。那些是模型里面最便宜的分数，而这个策略很彻底。

6 这和「规划器在决策时利用模型」有什么不同？

- a. 方案可以被查看和否决；而训练出来的策略走出来时，漏洞已经在它的权重里了
- b. 规划器不受模型误差影响
- c. 策略事后更容易纠正
- d. 是同一个问题换了个名字

答案 a. 方案可以被查看和否决；而训练出来的策略走出来时，漏洞已经在它的权重里了。正是这个差别让梦里那个版本更难被抓住。没有一份摆在那儿可以看的方案。

7 为什么要给一个你费劲做准的模型故意加上不确定性？

- a. 为了减少内存占用
- b. 为了把模型做小
- c. 为了加快训练
- d. 好让一个只有在模型碰巧那样掷一次时才管用的把戏，不再值得往上搭房子

答案 d. 好让一个只有在模型碰巧那样掷一次时才管用的把戏，不再值得往上搭房子。如果策略在利用某个怪癖，解法是让这个怪癖变得不可靠，而不是把模型做得更好。

8 不确定性旋钮的两端都会失败。怎么个失败法？

- a. 两端都让训练不稳
- b. 笃定的梦会被利用；噪声太大的梦里已经没有任务可学了
- c. 低端慢，高端快
- d. 只有高端会失败

答案 b. 笃定的梦会被利用；噪声太大的梦里已经没有任务可学了。有用的设置是一个峰而不是一个方向：那条「两种失败都还没占上风」的窄带。

FIG. 6.11 八道关于这个循环以及住在里面的东西的题。最后三道分开的是「卖点」和「这个循环实际交付的东西」。

上面每一个系统都默认了模型应该预测东西看起来是什么样：帧、像素、场景，你人在那儿会看到的東西。下一章问的是，这是不是正确的那份工作。

参考资料

World Models HA & SCHMIDHUBER, 2018 ↗

编码器、潜在动力学、紧凑控制器，还包括把策略完全放在模型生成的环境里训练、再搬回真实环境的实验。

<https://arxiv.org/abs/1803.10122>

Dream to Control (Dreamer) HAFNER ET AL., 2019 ↗

在想象出来的潜在轨迹上训练 actor 和 critic，于是动手的那一刻不必再跑昂贵的搜索。

<https://arxiv.org/abs/1912.01603>

Mastering Atari with Discrete World Models (DreamerV2) HAFNER ET AL., 2020 ↗

同一个循环，做到了它开始打败那些直接从环境里学的智能体的规模。

<https://arxiv.org/abs/2010.02193>

Mastering diverse control tasks through world models (DreamerV3)

HAFNER ET AL., NATURE 2025 ↗

同一个算法、同一套超参数，跑了 150 多个任务。这是想象这一支目前最强的证据。

<https://www.nature.com/articles/s41586-025-08744-2>

Dyna: an Integrated Architecture for Learning, Planning and Reacting

SUTTON, 1991 ↗

把规划定义成在学出来的模型上做计算，以及那个把它和真实经验交织起来的循环。

<https://mlanthology.org/icml/1990/sutton1990icml-integrated/>

When to Trust Your Model: Model-Based Policy Optimisation

JANNER ET AL., 2019 ↗

从真实状态出发的短推演，直接回应累积误差和模型利用。标题就是这一章的问题。

<https://arxiv.org/abs/1906.08253>

Domain Randomization for Transferring Deep Neural Networks

TOBIN ET AL., 2017 ↗

同一个想法从机器人学那边过来：让模拟器的变化比现实还大，于是没有任何东西能依赖它的某一个版本。

<https://arxiv.org/abs/1703.06907>

Solving Rubik's Cube with a Robot Hand OPENAI ET AL., 2019 ↗

把随机化推到足以把策略从模拟搬到真实硬件上，以及对这件事代价的一份诚实交代。

<https://arxiv.org/abs/1910.07113>

FIG. 6.12 排序是给要在这上面接着做事的人用的，而不是为了历史完整性，而且全程优先第一手材料。